

Lesson End Project

Integrating Penetration Testing and SQL Injection in DevSecOps with GitHub Actions

Project agenda: To implement penetration testing and SQL injection detection in a DevSecOps pipeline using GitHub Actions, enhancing the security of a Java application by automating security scans and generating vulnerability reports

Description: As a DevOps engineer, your task is to enhance the security of a deliberately vulnerable application by incorporating penetration testing and SQL injection detection into a GitHub Actions workflow. This involves setting up workflows to automatically perform penetration scans using OWASP ZAP, detect vulnerabilities, and log results as GitHub issues, ensuring early identification of potential threats.

Tools required: GitHub Actions and OWASP ZAP

Prerequisites: A GitHub account, an AWS account to launch an EC2 instance, basic knowledge of Docker, OWASP ZAP, SQL injection techniques, and YAML syntax for GitHub Actions workflows

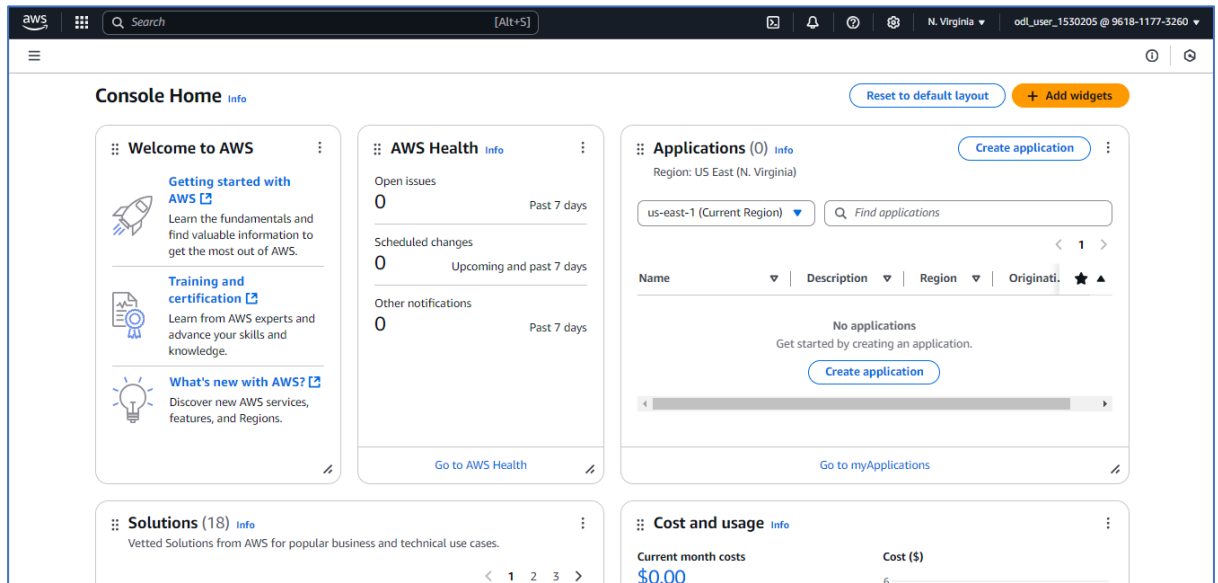
Expected deliverables: A GitHub Actions workflow for automated penetration testing using OWASP ZAP, a deployed WebGoat application on the EC2 instance, a baseline scan report identifying vulnerabilities like SQL injection, GitHub issues logged for detected vulnerabilities, and a consolidated security report in the repository

Steps to be followed:

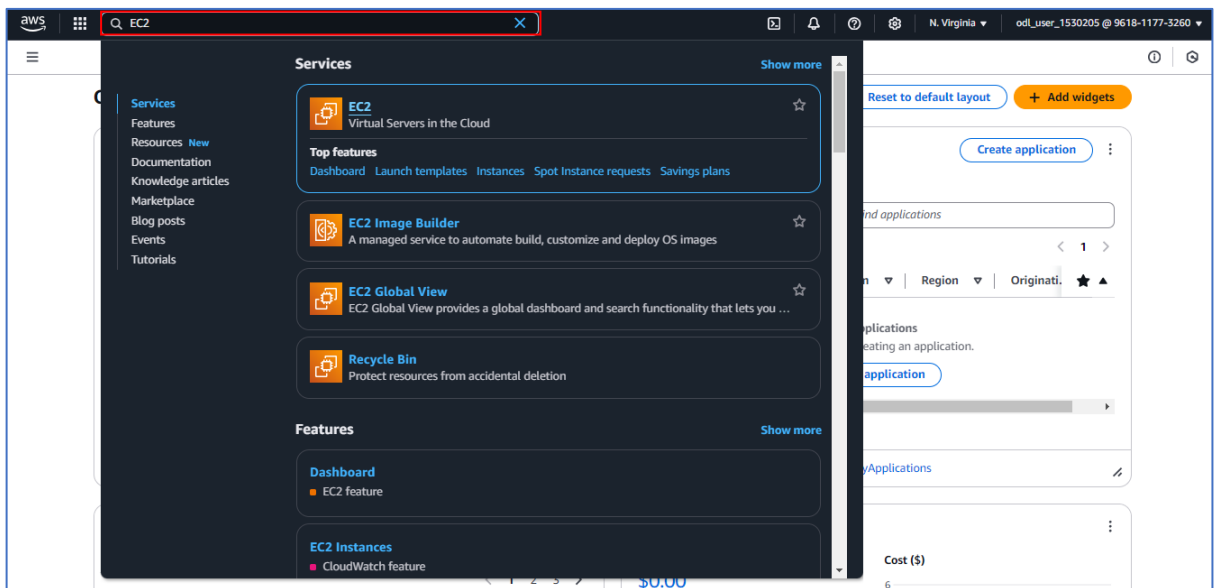
1. Set up EC2 Instance
2. Deploy the vulnerable application
3. Test SQL Injection
4. Create GitHub repository and set up Actions Workflow
5. Run Workflow and review results

Step 1: Set up EC2 Instance

1.1 Open the Amazon console at <https://console.aws.amazon.com/>

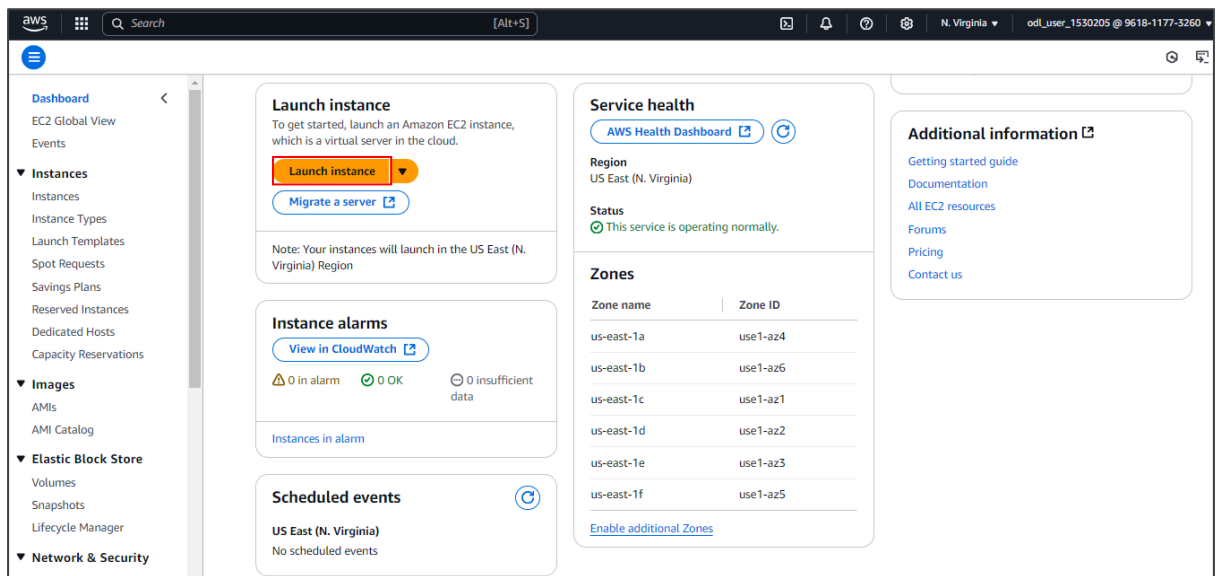


1.2 Type **EC2** in the search bar and press enter to navigate to EC2 Dashboard

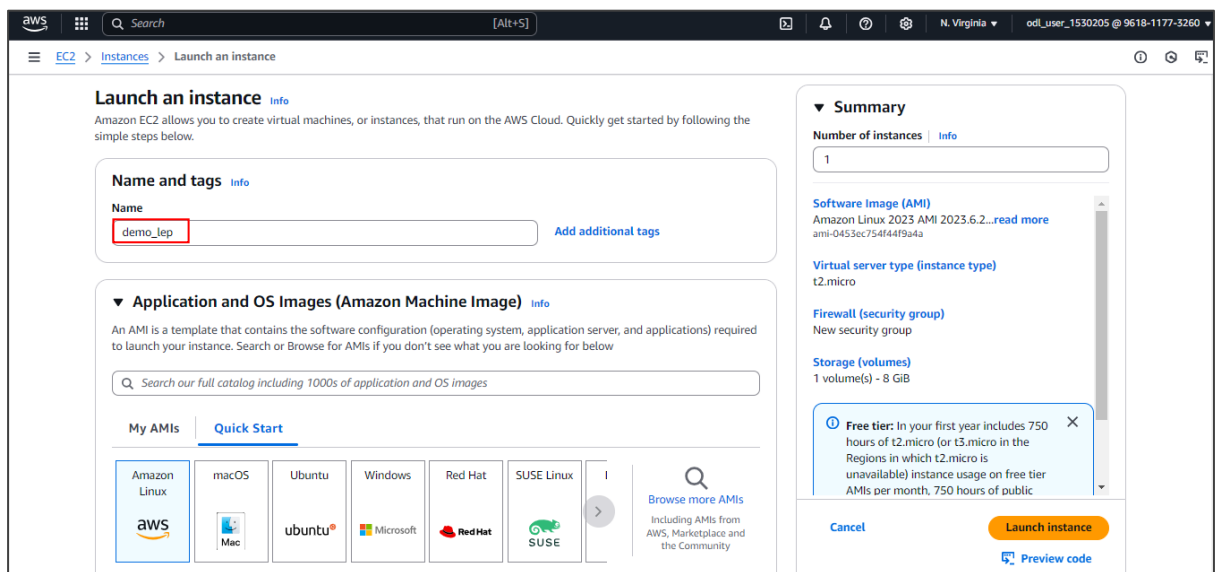


You will be redirected to the EC2 Dashboard.

1.3 Click on Launch instance



1.4 Add a desired name for the Instance (demo_lep)



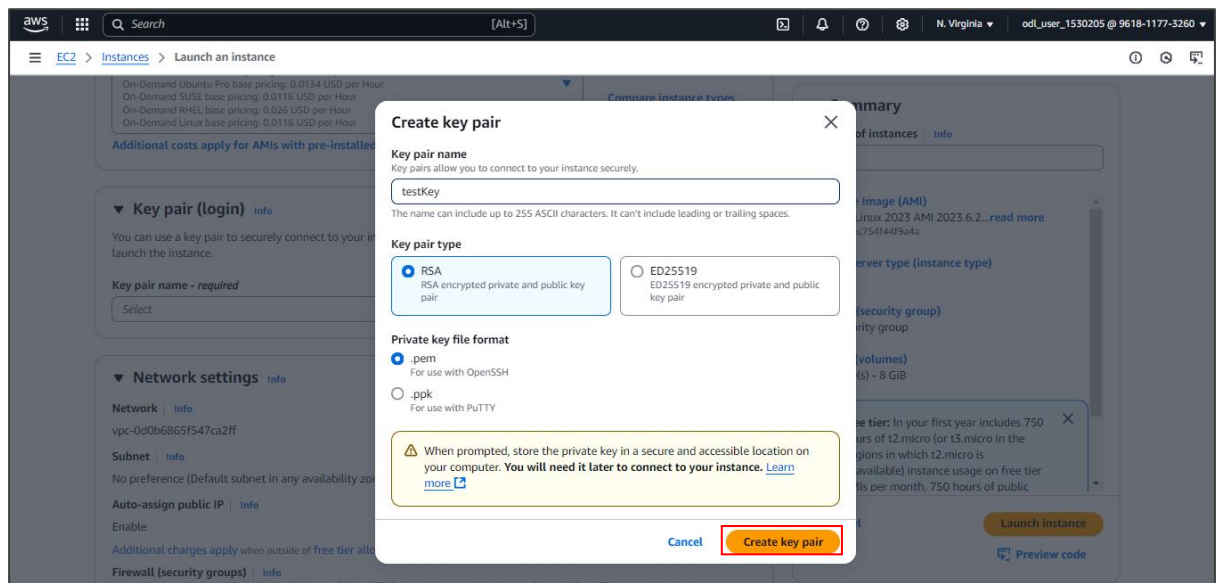
1.5 Select Amazon Linux

The screenshot shows the AWS Management Console 'Launch an instance' page. The 'Amazon Linux' option is highlighted with a red box in the top navigation bar. Below the navigation bar, the 'Amazon Machine Image (AMI)' section displays 'Amazon Linux 2023 AMI' with details: ami-0453ec754f44f9a4a (64-bit (x86), uefi-preferred) / ami-0ed83e7a78a23014e (64-bit (Arm), uefi). The 'Description' section states that Amazon Linux 2023 is a modern, general purpose Linux-based OS. The 'Architecture' is set to '64-bit (x86)' and the 'Boot mode' is 'uefi-preferred'. The 'AMI ID' is 'ami-0453ec754f44f9a4a' and the 'Username' is 'ec2-user'. A 'Verified provider' badge is visible. The 'Summary' panel on the right shows 'Number of instances' as 1, 'Software Image (AMI)' as 'Amazon Linux 2023 AMI 2023.6.2...', 'Virtual server type (instance type)' as 't2.micro', 'Firewall (security group)' as 'New security group', and 'Storage (volumes)' as '1 volume(s) - 8 GiB'. A 'Free tier' notification indicates 750 hours of t2.micro usage. The 'Launch instance' button is highlighted in orange.

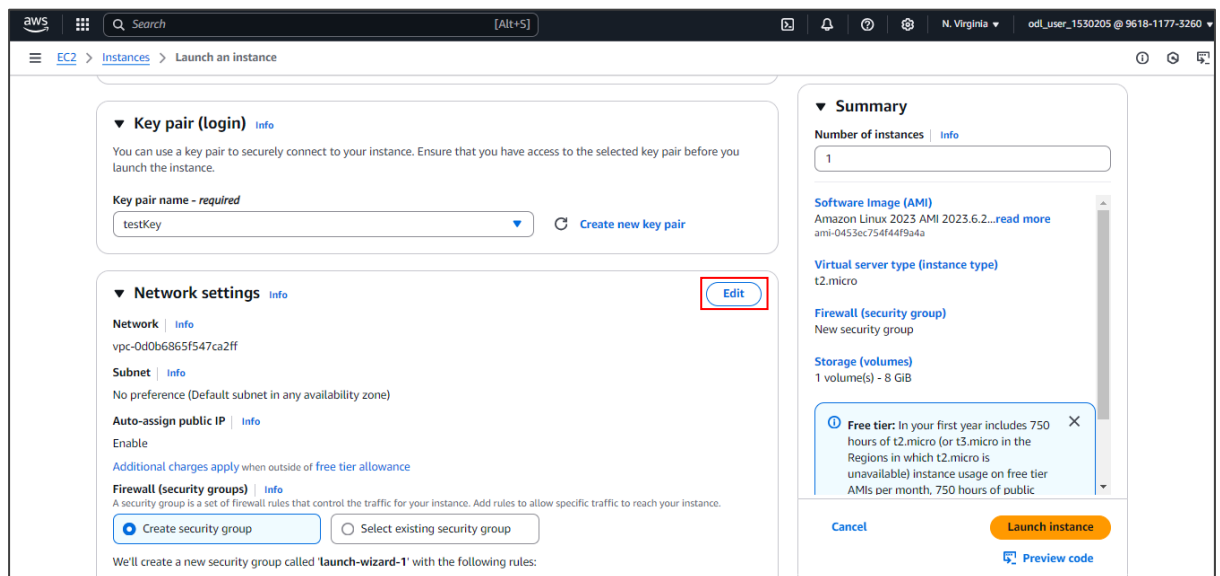
1.6 Scroll down and click on Create new key pair

The screenshot shows the AWS Management Console 'Launch an instance' page, scrolled down to the 'Key pair (login)' section. The 'Key pair name - required' dropdown menu is set to 'Select'. The 'Create new key pair' button is highlighted with a red box. The 'Network settings' section shows 'Network' as 'vpc-0d0b6865f547ca2ff', 'Subnet' as 'No preference (Default subnet in any availability zone)', and 'Auto-assign public IP' as 'Enable'. The 'Summary' panel on the right shows 'Number of instances' as 1, 'Software Image (AMI)' as 'Amazon Linux 2023 AMI 2023.6.2...', 'Virtual server type (instance type)' as 't2.micro', 'Firewall (security group)' as 'New security group', and 'Storage (volumes)' as '1 volume(s) - 8 GiB'. A 'Free tier' notification indicates 750 hours of t2.micro usage. The 'Launch instance' button is highlighted in orange.

1.7 Add a desired name for the key pair and click on **Create key pair**



1.8 Scroll down and click on **Edit** in the Network settings section



1.9 Scroll down and click **Add security group rule**

Security group name - *required*
launch-wizard-1

Description - *required* | [Info](#)
launch-wizard-1 created 2024-12-01T13:07:34.760Z

Inbound Security Group Rules
▼ Security group rule 1 (TCP, 22, 0.0.0.0/0) [Remove](#)

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Source Info	Description - <i>optional</i> Info
Anywhere	Add CIDR, prefix list or security group 0.0.0.0/0	e.g. SSH for admin desktop

[Add security group rule](#)

▼ Configure storage | [Info](#) [Advanced](#)

1x 8 GiB gp3 Root volume (Not encrypted)

▼ Summary

Number of instances | [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.6.2...[read more](#)
ami-0453ec754f44f9a4a

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year includes 750 hours of t2.micro (or t3.micro in the Regions in which t2.micro is unavailable) instance usage on free tier AMIs per month, 750 hours of public

[Cancel](#) [Launch instance](#) [Preview code](#)

1.10 Select **All traffic** under the Type selector and **Anywhere** under the Source type selector

Type | [Info](#)

ssh

Source type | [Info](#)

Anywhere

Protocol | [Info](#)

TCP

Port range | [Info](#)

22

Description - *optional* | [Info](#)

e.g. SSH for admin desktop

▼ Security group rule 2 (All, All, 0.0.0.0/0) [Remove](#)

Type Info	Protocol Info	Port range Info
All traffic	All	All
Source type Info	Source Info	Description - <i>optional</i> Info
Anywhere	Add CIDR, prefix list or security group 0.0.0.0/0	e.g. SSH for admin desktop

[Add security group rule](#)

▼ Configure storage | [Info](#) [Advanced](#)

1x 8 GiB gp3 Root volume (Not encrypted)

▼ Summary

Number of instances | [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.6.2...[read more](#)
ami-0453ec754f44f9a4a

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year includes 750 hours of t2.micro (or t3.micro in the Regions in which t2.micro is unavailable) instance usage on free tier AMIs per month, 750 hours of public

[Cancel](#) [Launch instance](#) [Preview code](#)

1.11 Click on Launch instance

The screenshot shows the AWS Management Console interface for launching an EC2 instance. The configuration is as follows:

- Type:** ssh
- Protocol:** TCP
- Port range:** 22
- Source type:** Anywhere
- Source:** 0.0.0.0/0
- Description - optional:** e.g. SSH for admin desktop
- Security group rule 2:** All traffic, All, 0.0.0.0/0
- Configure storage:** 1x 8 GiB gp3 Root volume (Not encrypted)

The **Summary** panel on the right indicates:

- Number of instances:** 1
- Software Image (AMI):** Amazon Linux 2023 AMI 2023.6.2...
- Virtual server type (instance type):** t2.micro
- Firewall (security group):** New security group
- Storage (volumes):** 1 volume(s) - 8 GiB

A **Free tier** notification states: "In your first year includes 750 hours of t2.micro (or t3.micro in the Regions in which t2.micro is unavailable) instance usage on free tier AMIs per month, 750 hours of public".

The **Launch instance** button is highlighted with a red box.

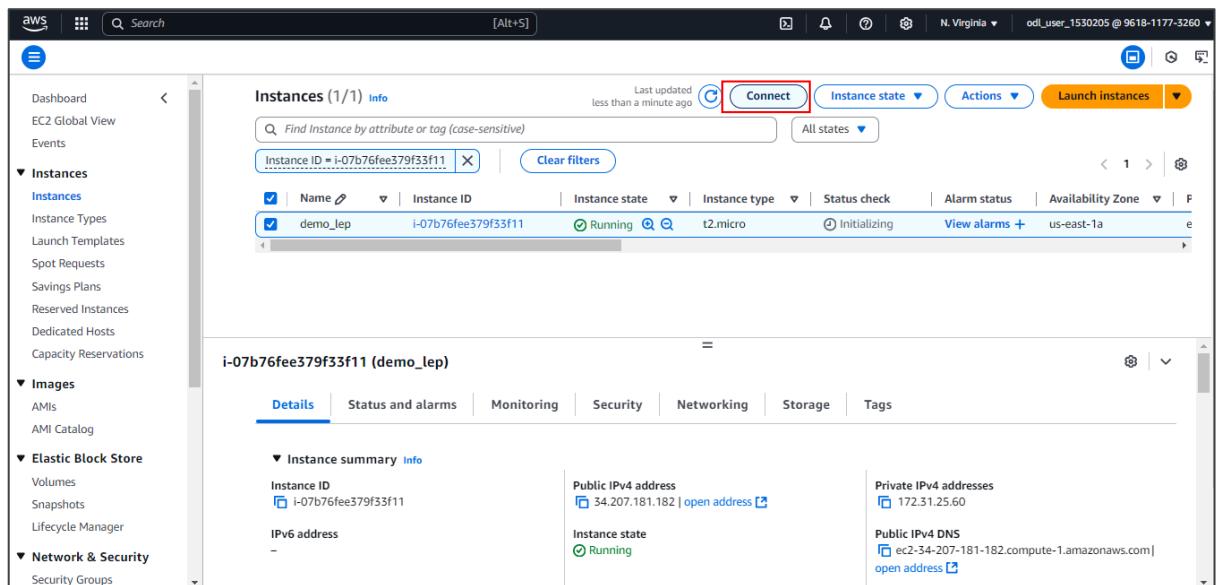
1.12 Select the instance created from the message box

The screenshot shows the AWS Management Console interface after successfully launching an EC2 instance. A green banner at the top indicates: "Success Successfully initiated launch of instance i-07b76fee379f33f11".

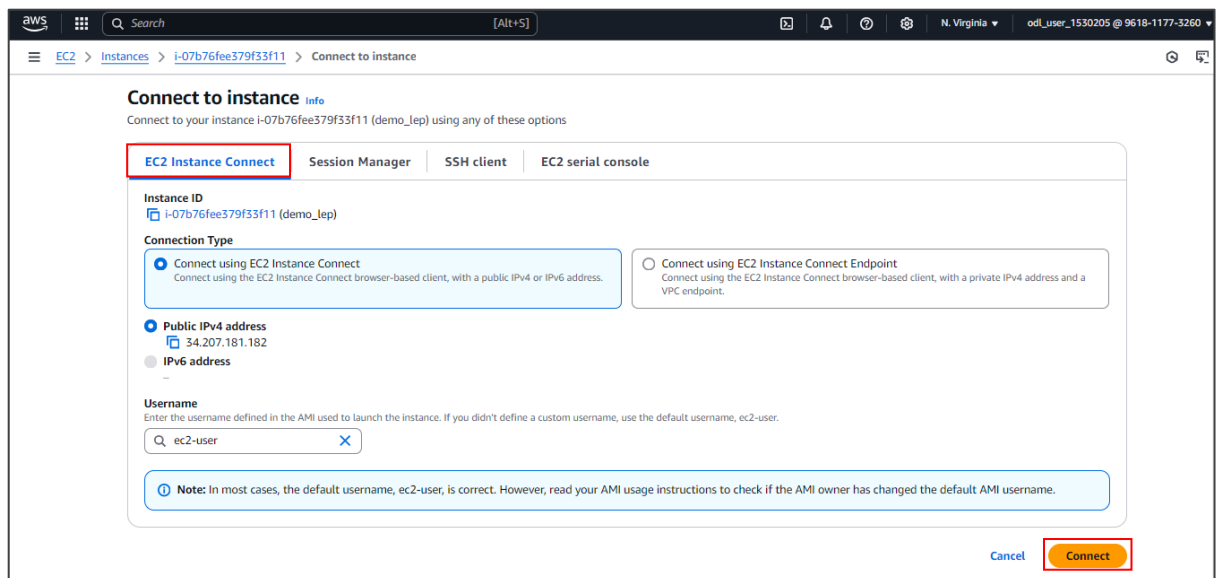
The **Next Steps** section provides various actions to take with the new instance:

- Create billing and free tier usage alerts:** To manage costs and avoid surprise bills, set up email notifications for billing and free tier usage thresholds. [Create billing alerts](#)
- Connect to your instance:** Once your instance is running, log into it from your local computer. [Connect to instance](#)
- Connect an RDS database:** Configure the connection between an EC2 instance and a database to allow traffic flow between them. [Connect an RDS database](#)
- Create EBS snapshot policy:** Create a policy that automates the creation, retention, and deletion of EBS snapshots. [Create EBS snapshot policy](#)
- Manage detailed monitoring:** Enable or disable detailed monitoring for the instance.
- Create Load Balancer:** Create an application, network, gateway, or classic Load Balancer.
- Create AWS budget:** AWS Budgets allows you to create budgets.
- Manage CloudWatch alarms:** Create or update Amazon CloudWatch alarms.

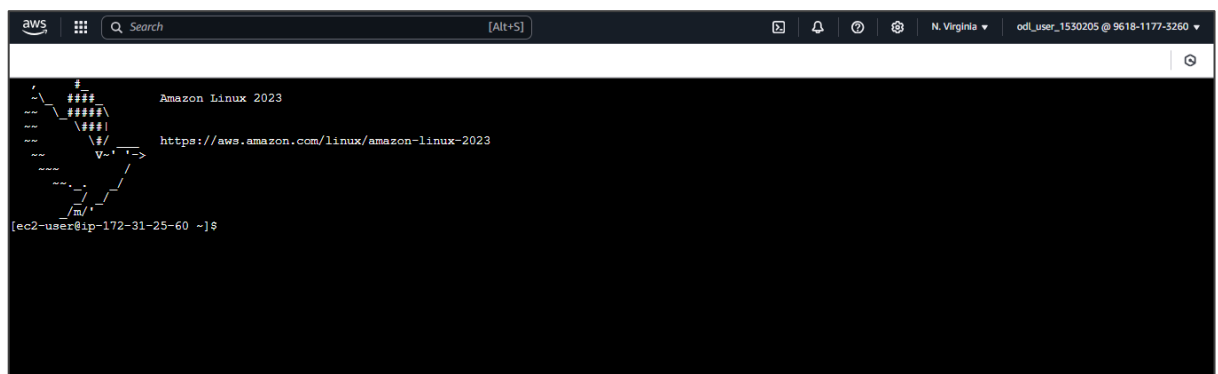
1.13 Select the instance created and click on **Connect**



1.14 Select **EC2 Instance Connect**. Then, scroll down and click on **Connect**.



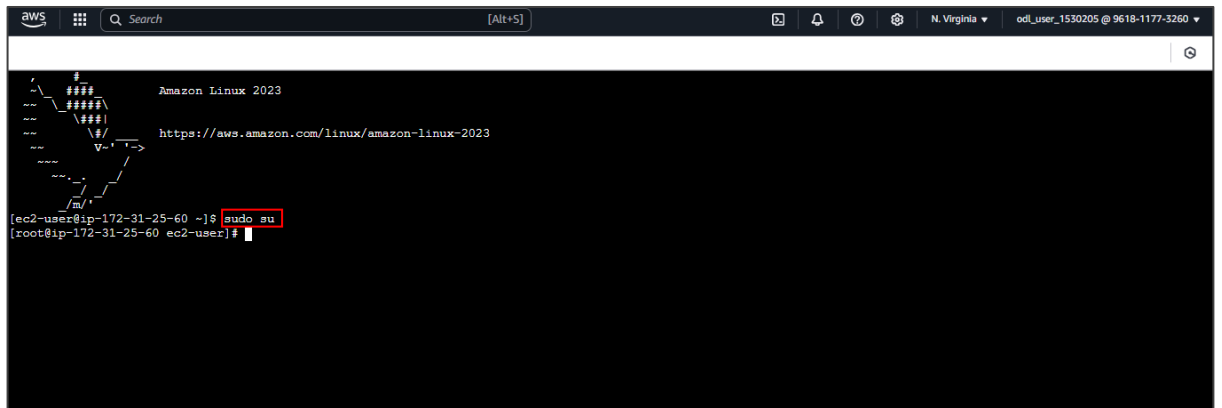
Now, you will be redirected to the EC2 terminal.



Step 2: Deploy the vulnerable application

2.1 Run the following command to switch to the root user:

sudo su



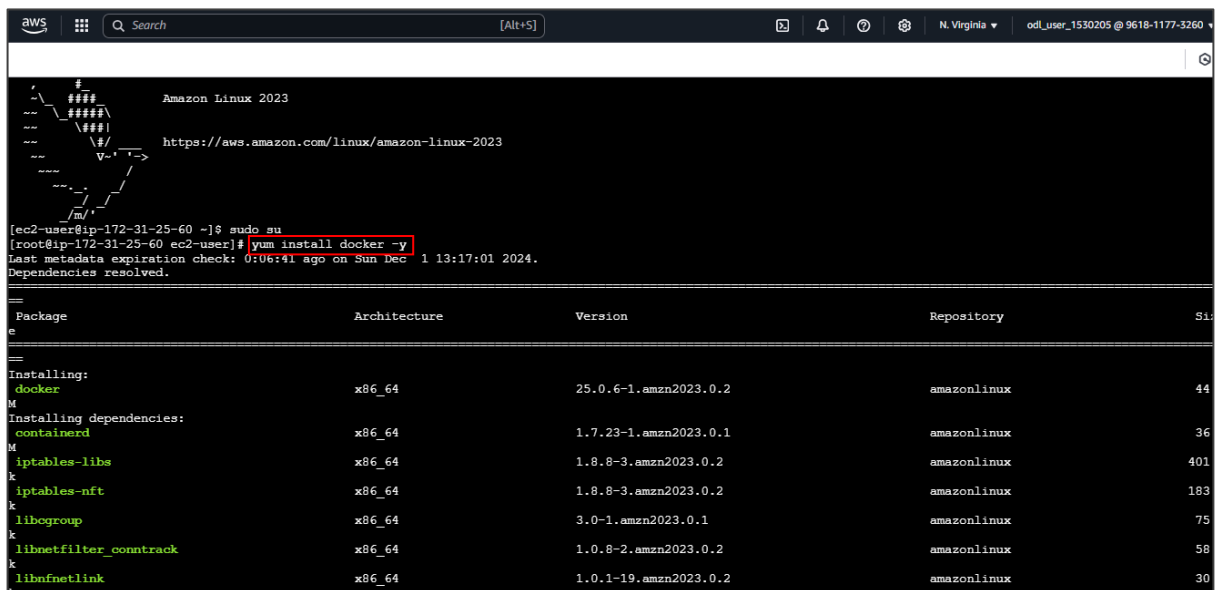
```
aws [Alt+S] N. Virginia odL_user_1530205 @ 9618-1177-3260

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-25-60 ~]$ sudo su
[root@ip-172-31-25-60 ec2-user]#
```

2.2 Run the following command to install docker:

yum install docker -y



```
aws [Alt+S] N. Virginia odL_user_1530205 @ 9618-1177-3260

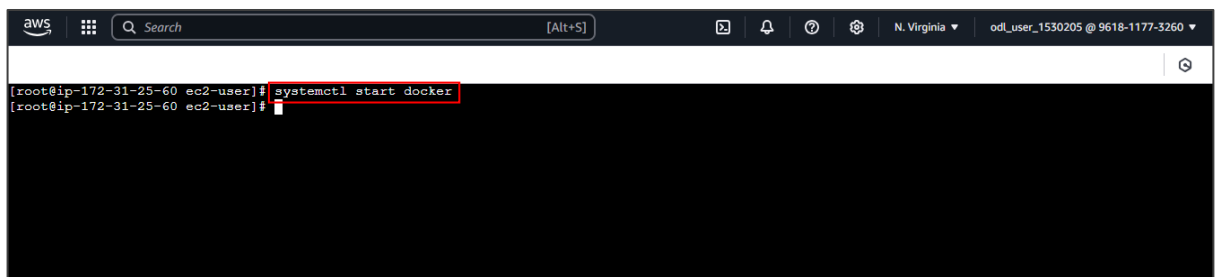
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-25-60 ~]$ sudo su
[root@ip-172-31-25-60 ec2-user]# yum install docker -y
Last metadata expiration check: 0:06:41 ago on Sun Dec 1 13:17:01 2024.
Dependencies resolved.

=====
Package                                Architecture      Version            Repository          Size
=====
Installing:
docker                                x86_64            25.0.6-1.amzn2023.0.2  amazonlinux         44
Installing dependencies:
containerd                            x86_64            1.7.23-1.amzn2023.0.1  amazonlinux         36
iptables-libs                         x86_64            1.8.8-3.amzn2023.0.2  amazonlinux         401
iptables-nft                         x86_64            1.8.8-3.amzn2023.0.2  amazonlinux         183
libcgroup                             x86_64            3.0-1.amzn2023.0.1    amazonlinux         75
libnetfilter_conntrack                x86_64            1.0.8-2.amzn2023.0.2  amazonlinux         58
libnftnl                             x86_64            1.0.1-19.amzn2023.0.2  amazonlinux         30
=====
```

2.3 Run the following command:

systemctl start docker

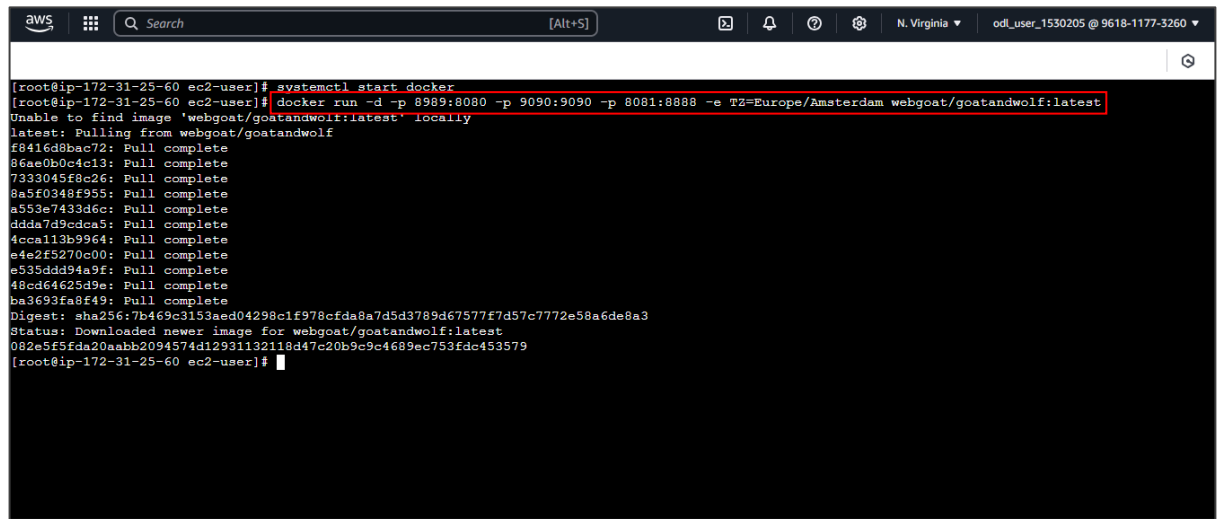


```
aws [Alt+S] N. Virginia odL_user_1530205 @ 9618-1177-3260

[root@ip-172-31-25-60 ec2-user]# systemctl start docker
[root@ip-172-31-25-60 ec2-user]#
```

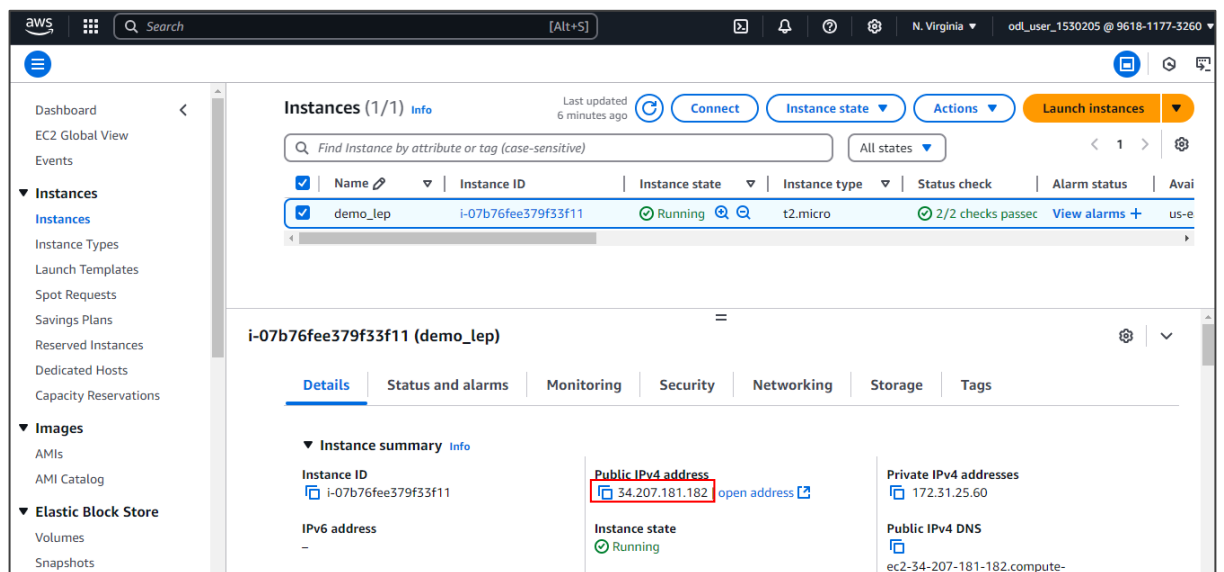
2.4 Run the following command to deploy the webgoat application:

```
docker run -d -p 8989:8080 -p 9090:9090 -p 8081:8888 -e TZ=Europe/Amsterdam  
webgoat/goatandwolf:latest
```



```
[root@ip-172-31-25-60 ec2-user]# systemctl start docker
[root@ip-172-31-25-60 ec2-user]# docker run -d -p 8989:8080 -p 9090:9090 -p 8081:8888 -e TZ=Europe/Amsterdam webgoat/goatandwolf:latest
Unable to find image 'webgoat/goatandwolf:latest' locally
latest: Pulling from webgoat/goatandwolf
f8416d8bac72: Pull complete
86ae0b0c4c13: Pull complete
7333045f8c26: Pull complete
8a5f0348f955: Pull complete
a553e7433d6c: Pull complete
ddda7d9cdca5: Pull complete
4cca113b9964: Pull complete
e4e2f5270c00: Pull complete
e535ddd94a9f: Pull complete
48cd64625d9e: Pull complete
ba3693fa8f49: Pull complete
Digest: sha256:7b469c3153aed04298c1f978cfda8a7d5d3789d67577f7d57c7772e58a6de8a3
Status: Downloaded newer image for webgoat/goatandwolf:latest
082e5f5fda20aabb2094574d12931132118d47c20b9c9c4689ec753fde453579
[root@ip-172-31-25-60 ec2-user]#
```

2.5 Navigate to the EC2 instance and copy the Public IPv4 address



The screenshot shows the AWS Management Console interface. On the left, the navigation menu is visible with sections for Dashboard, EC2 Global View, Events, Instances, Images, and Elastic Block Store. The 'Instances' section is expanded, showing a list of instances. The instance 'demo_lep' with ID 'i-07b76fee379f33f11' is selected and its details are shown on the right. The 'Details' tab is active, displaying the 'Instance summary' section. In this section, the 'Public IPv4 address' is highlighted with a red box and is '34.207.181.182'. Other details include the Instance ID, Instance state (Running), and Public IPv4 DNS.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Avail
demo_lep	i-07b76fee379f33f11	Running	t2.micro	2/2 checks passed	View alarms +	us-e

i-07b76fee379f33f11 (demo_lep)

Instance summary

Instance ID	Public IPv4 address	Private IPv4 addresses
i-07b76fee379f33f11	34.207.181.182	172.31.25.60

Instance state

Running

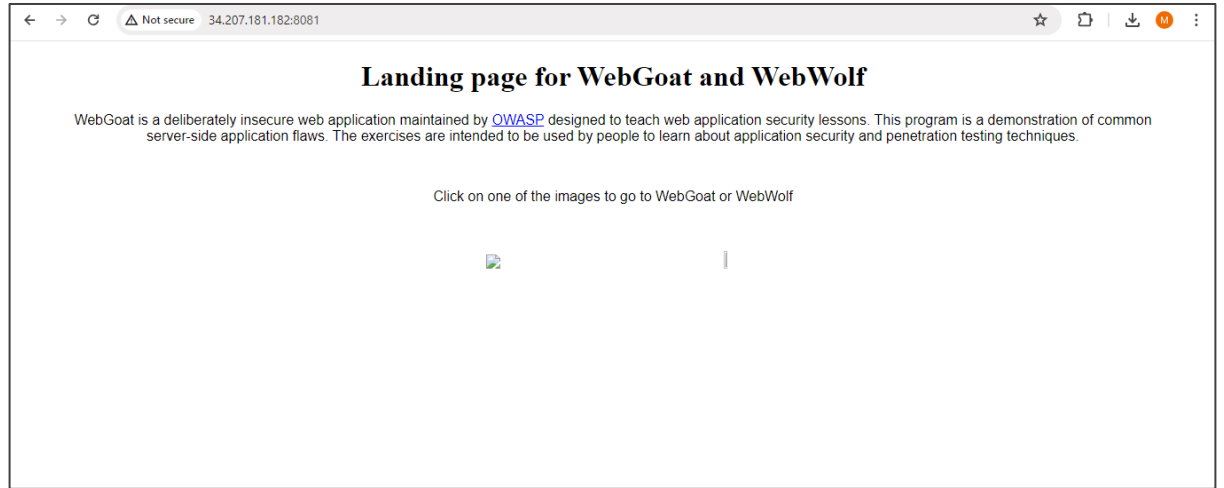
Public IPv4 DNS

ec2-34-207-181-182.compute-

Step 3: Test SQL Injection

3.1 Navigate to the browser and paste the following link:

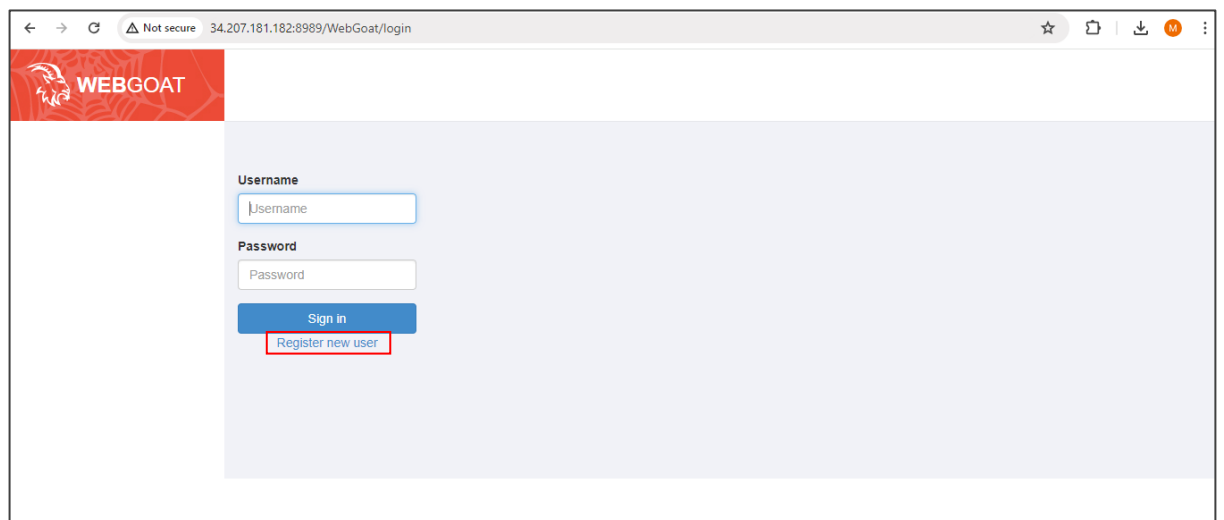
http://<publicIPServer>:8081



Note: Replace the <publicIPServer> with the Public IPv4 address copied

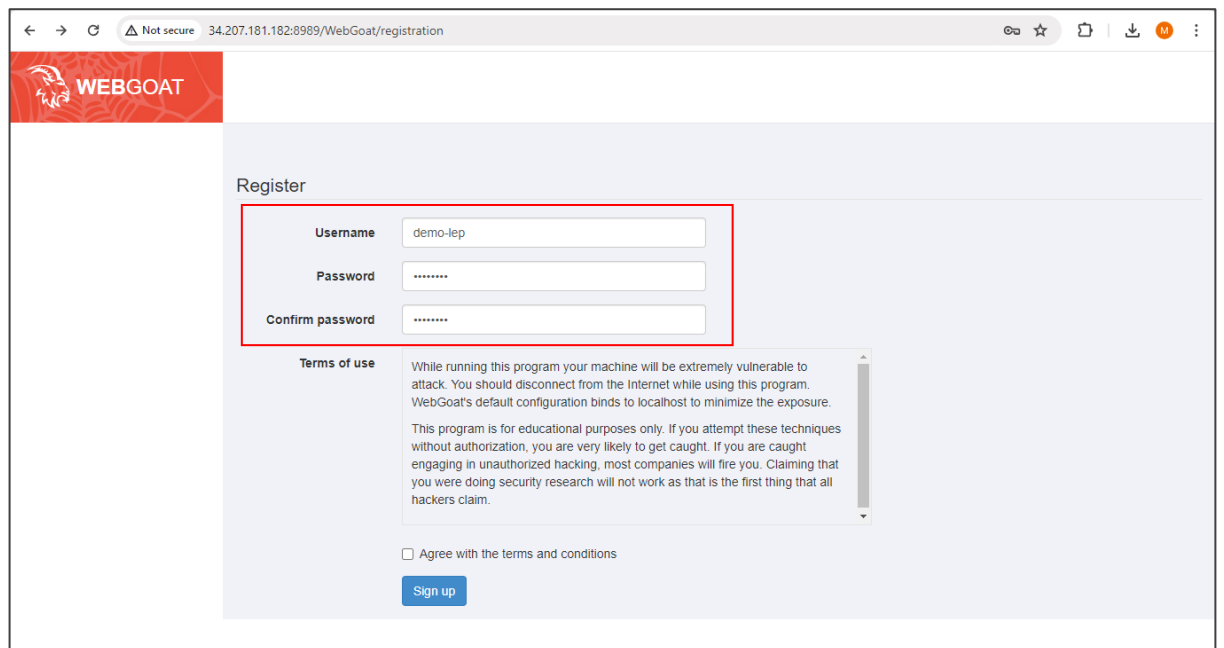
3.2 Paste the link given below in a new tab and click on **Register new user**

http://<PublicIPServer>:8989/WebGoat/login



Note: Replace the <publicIPServer> with the Public IPv4 address copied

3.3 Enter the Username, Password, and Confirm password as desired



The screenshot shows the WebGoat registration page in a web browser. The browser's address bar displays "34.207.181.182:8989/WebGoat/registration". The page features a red header with the "WEBGOAT" logo. The main content area is titled "Register" and contains a registration form. A red rectangular box highlights the "Username", "Password", and "Confirm password" input fields. The "Username" field contains the text "demo-lep". The "Password" and "Confirm password" fields are filled with asterisks. Below the input fields is a "Terms of use" section with a scrollable text area containing a disclaimer. At the bottom of the form, there is an unchecked checkbox labeled "Agree with the terms and conditions" and a blue "Sign up" button.

Register

Username demo-lep

Password

Confirm password

Terms of use

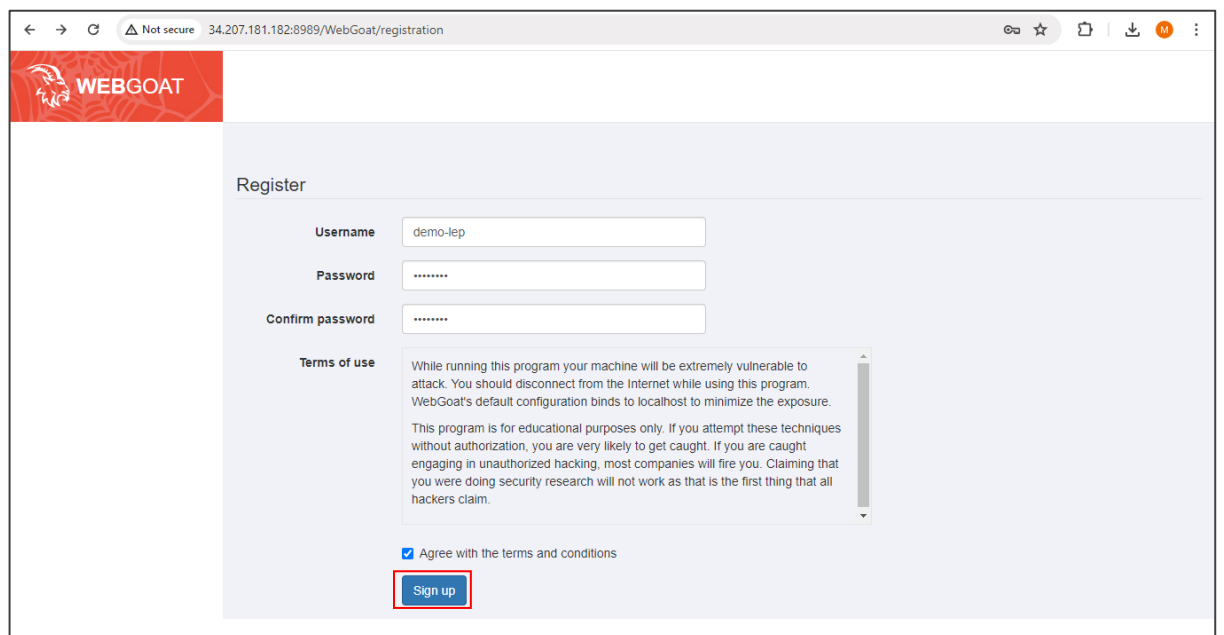
While running this program your machine will be extremely vulnerable to attack. You should disconnect from the Internet while using this program. WebGoat's default configuration binds to localhost to minimize the exposure.

This program is for educational purposes only. If you attempt these techniques without authorization, you are very likely to get caught. If you are caught engaging in unauthorized hacking, most companies will fire you. Claiming that you were doing security research will not work as that is the first thing that all hackers claim.

☐ Agree with the terms and conditions

Sign up

3.4 Select the checkbox and click on **Sign up**



This screenshot shows the same WebGoat registration page as the previous one, but with the "Agree with the terms and conditions" checkbox checked and the "Sign up" button highlighted by a red rectangular box. The "Username" field still contains "demo-lep", and the password fields remain masked with asterisks. The "Terms of use" section and the "Sign up" button are visible in the same positions as in the previous screenshot.

Register

Username demo-lep

Password

Confirm password

Terms of use

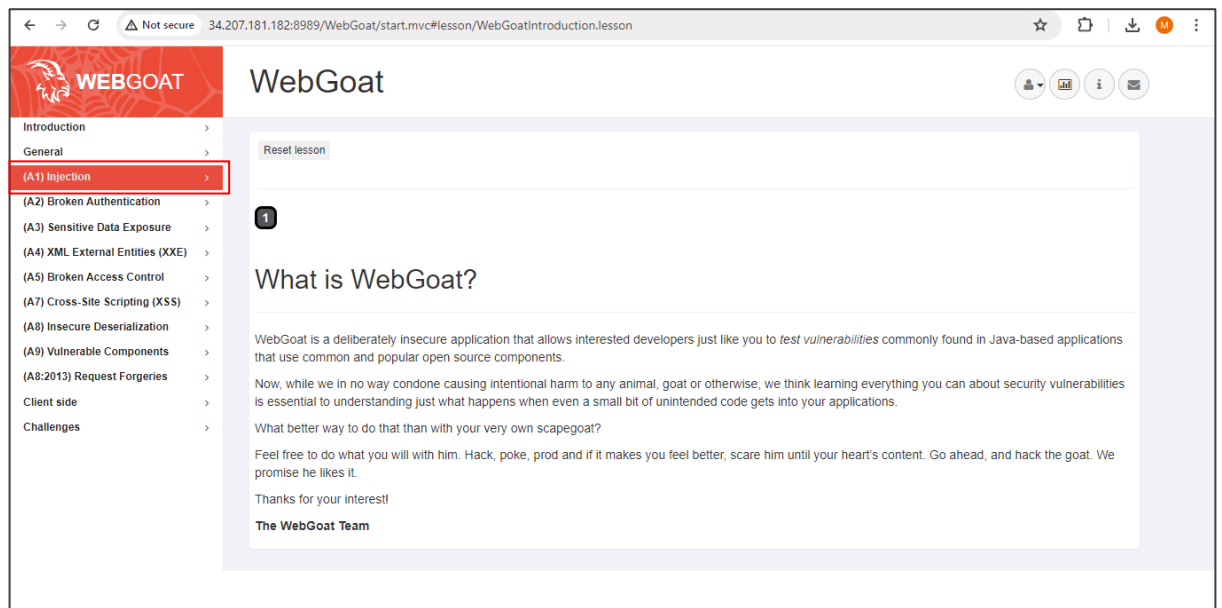
While running this program your machine will be extremely vulnerable to attack. You should disconnect from the Internet while using this program. WebGoat's default configuration binds to localhost to minimize the exposure.

This program is for educational purposes only. If you attempt these techniques without authorization, you are very likely to get caught. If you are caught engaging in unauthorized hacking, most companies will fire you. Claiming that you were doing security research will not work as that is the first thing that all hackers claim.

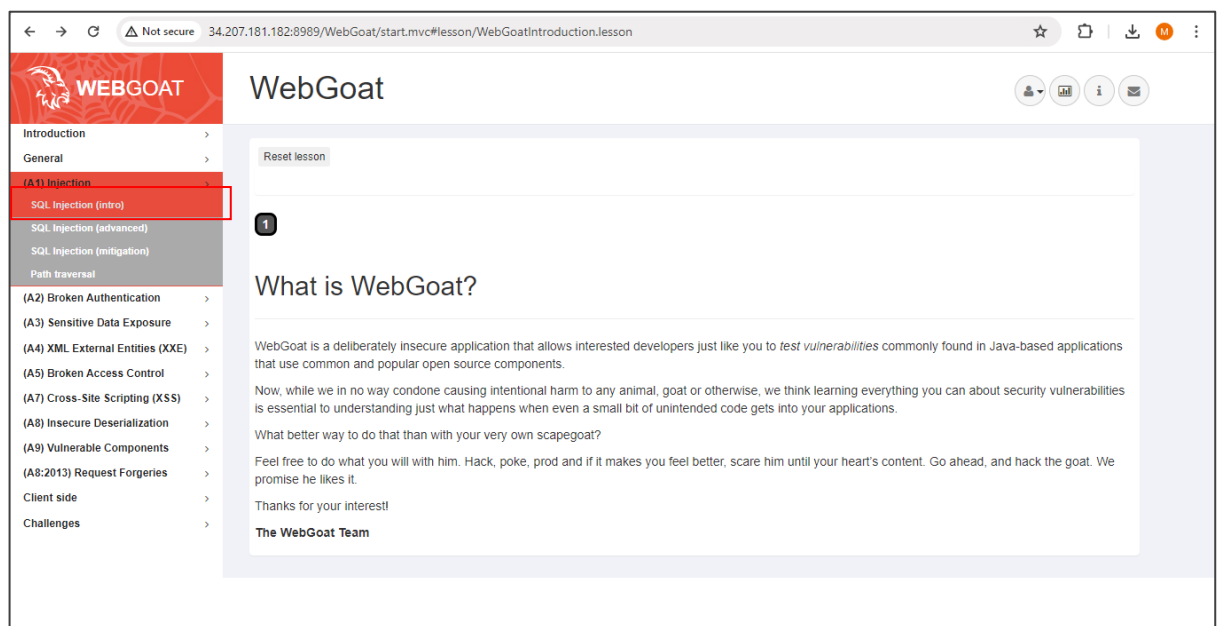
☒ Agree with the terms and conditions

Sign up

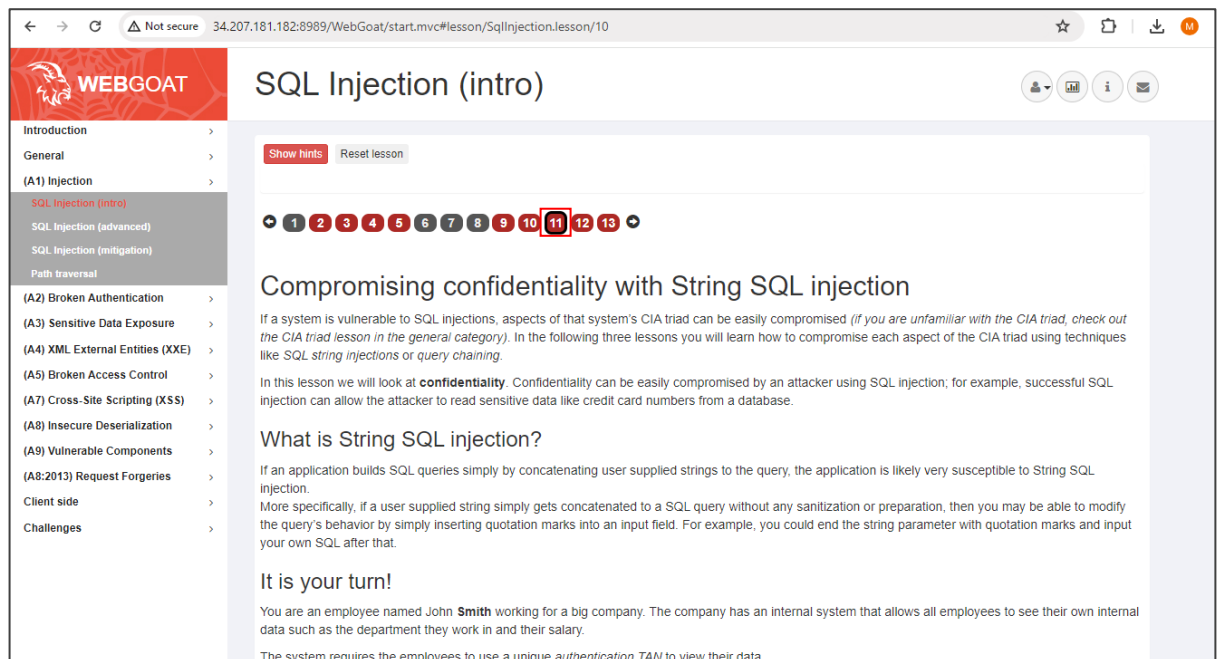
3.5 Click on (A1) Injection



3.6 Click on SQL Injection (intro)



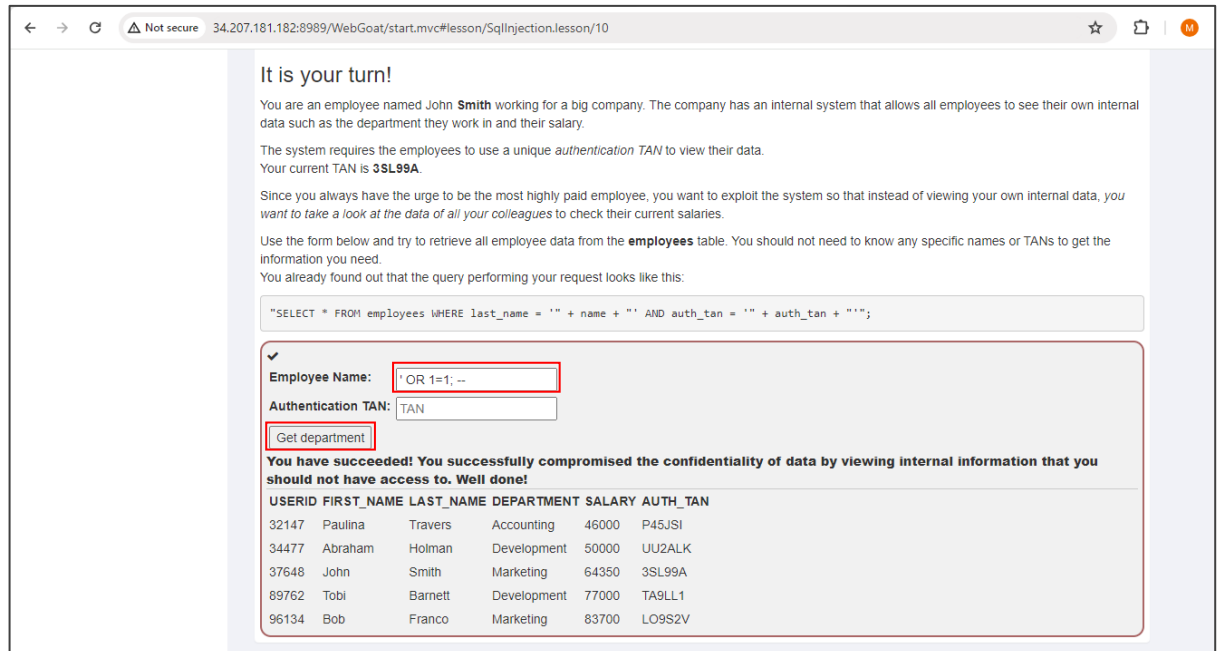
3.7 Select lesson 11 in the SQL Injection



The screenshot shows the WebGoat interface for the SQL Injection (intro) lesson. The left sidebar contains a navigation menu with various lessons. The main content area is titled "SQL Injection (intro)" and includes a progress bar with 13 lessons. Lesson 11 is highlighted. The lesson content is titled "Compromising confidentiality with String SQL injection" and includes an introduction to SQL injection, a definition of String SQL injection, and a section titled "It is your turn!" which describes a scenario where an employee named John Smith is working for a company with an internal system that allows all employees to see their own internal data such as the department they work in and their salary. The system requires the employees to use a unique authentication TAN to view their data.

3.8 Scroll down, enter the following text in the Employee Name, and click on the Get department button

' OR 1=1; --



The screenshot shows the "It is your turn!" section of the lesson. It includes a scenario where an employee named John Smith is working for a company with an internal system that allows all employees to see their own internal data such as the department they work in and their salary. The system requires the employees to use a unique authentication TAN to view their data. The current TAN is 3SL99A. The user is instructed to use the form below to try to retrieve all employee data from the employees table. The form has two fields: "Employee Name" and "Authentication TAN". The "Employee Name" field contains the text ' OR 1=1; --. The "Authentication TAN" field contains the text TAN. The "Get department" button is highlighted. Below the form, a message states: "You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!". Below this message is a table showing employee data.

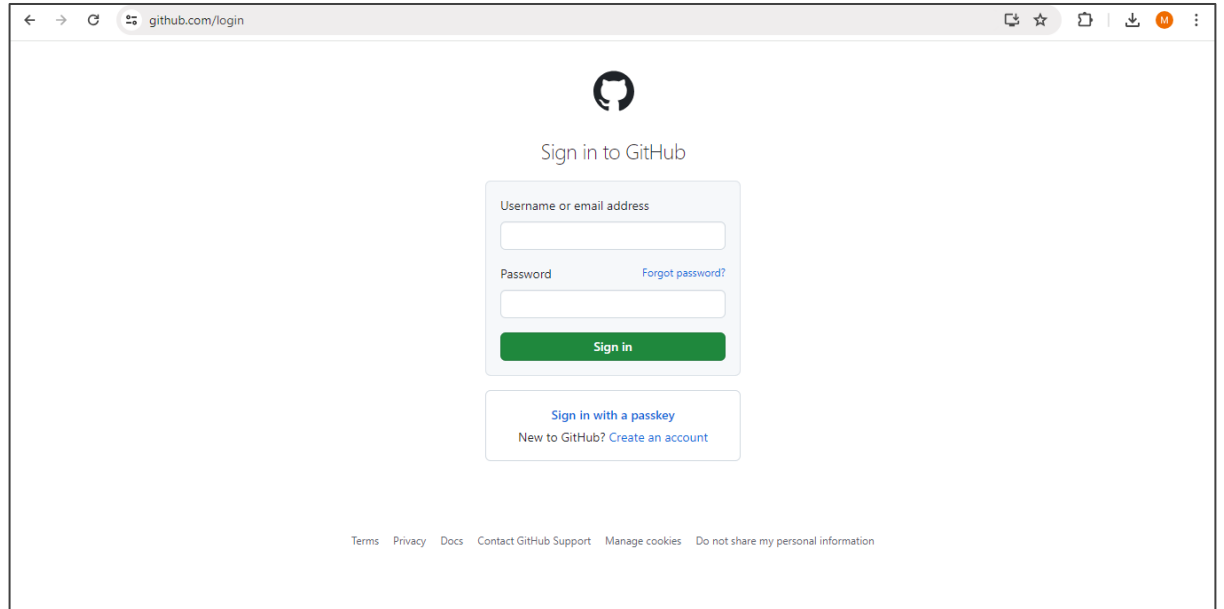
USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
32147	Paulina	Travers	Accounting	46000	P45JSI
34477	Abraham	Holman	Development	50000	UU2ALK
37648	John	Smith	Marketing	64350	3SL99A
89762	Tobi	Barnett	Development	77000	TA9LL1
96134	Bob	Franco	Marketing	83700	LO9S2V

Now, you can see the table data instead of an error indicating the site is vulnerable.

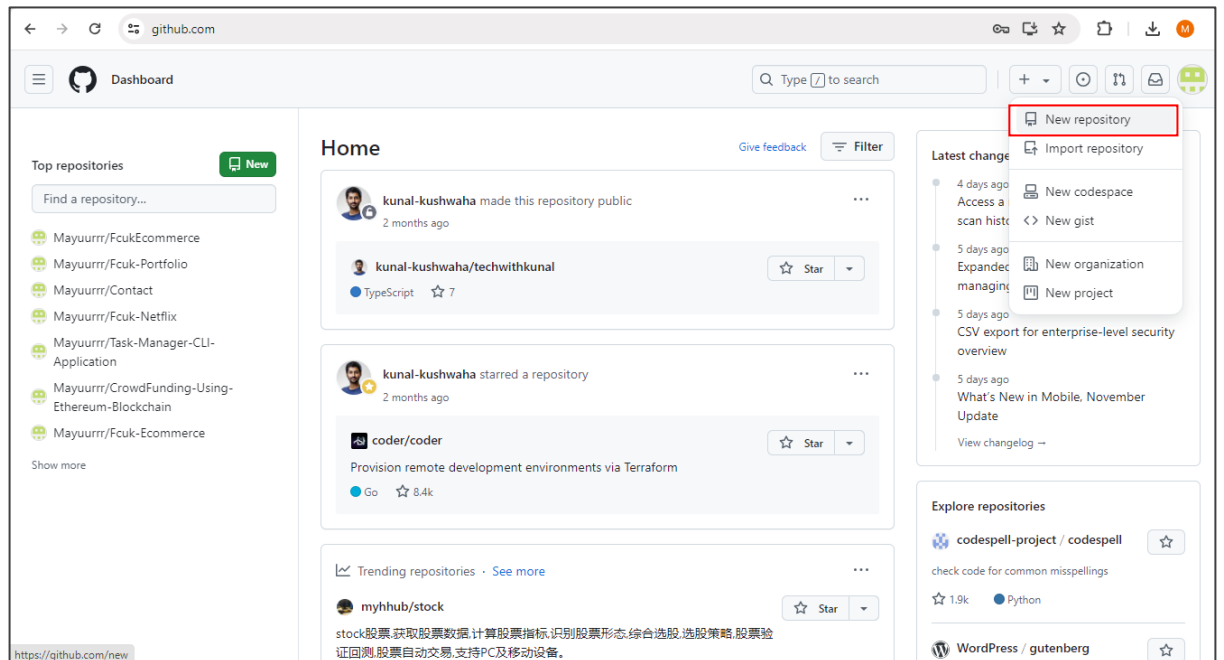
Step 4: Create GitHub repository and set up Actions Workflow

4.1 Sign in to your GitHub account using the following URL:

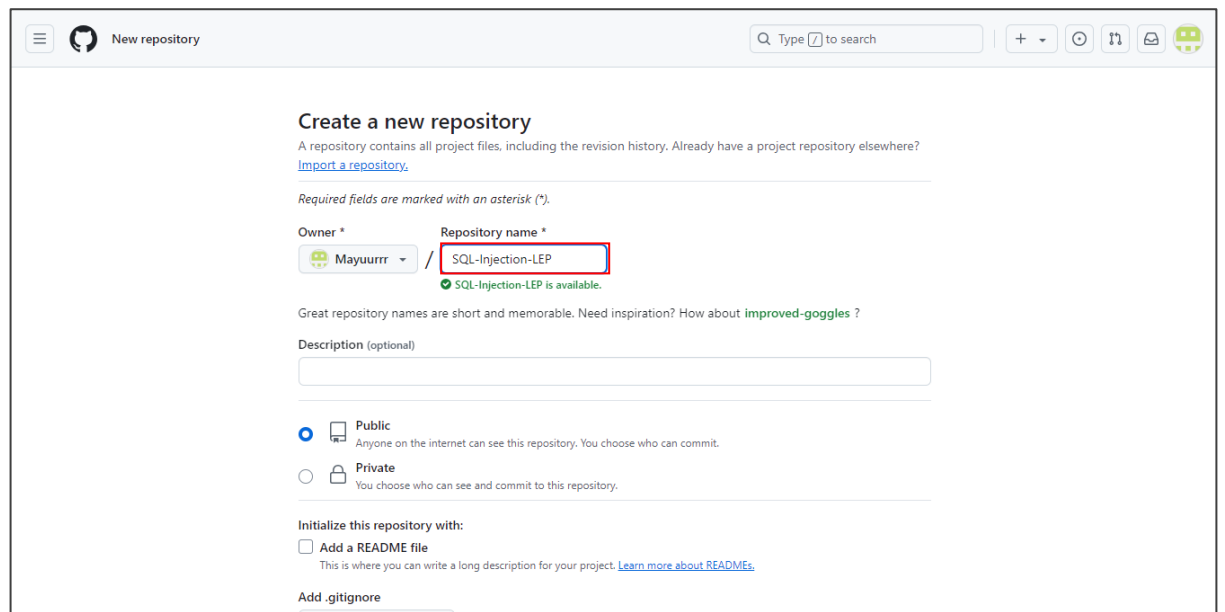
<https://github.com/login>



4.2 Click on **New repository**



4.3 Enter the repository name as desired



New repository

Search Type [] to search

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * Mayuurr / Repository name * SQL-Injection-LEP

SQL-Injection-LEP is available.

Great repository names are short and memorable. Need inspiration? How about [improved-goggles](#)?

Description (optional)

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

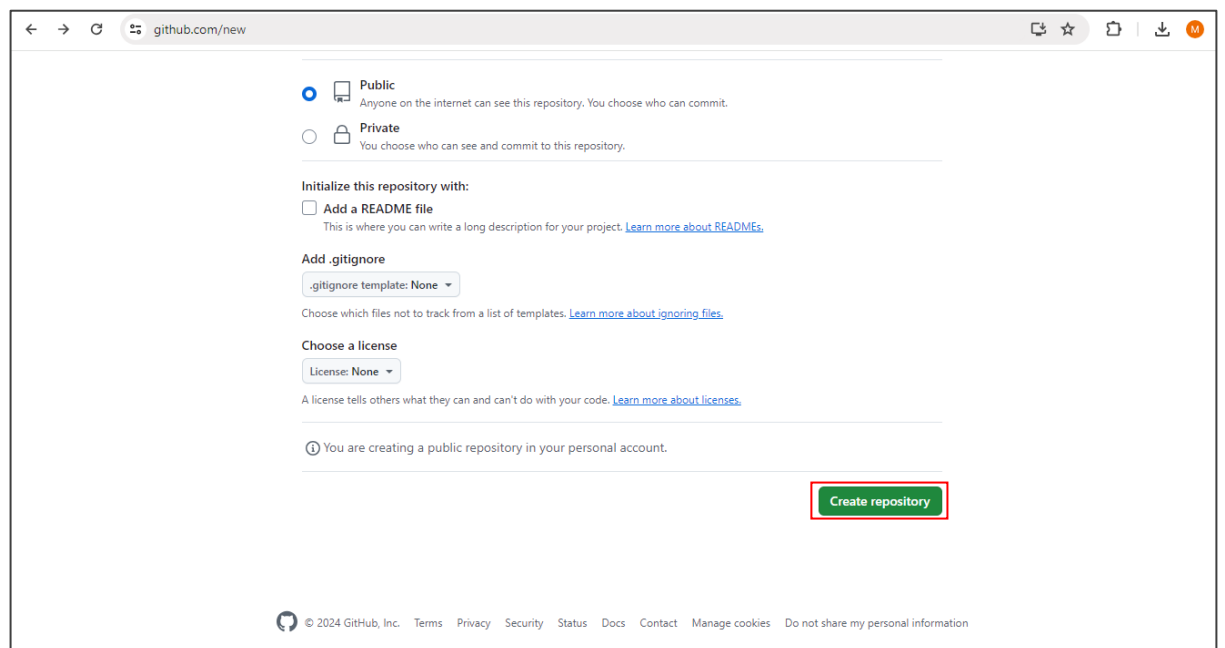
☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

4.4 Scroll down and click on **Create repository**



github.com/new

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

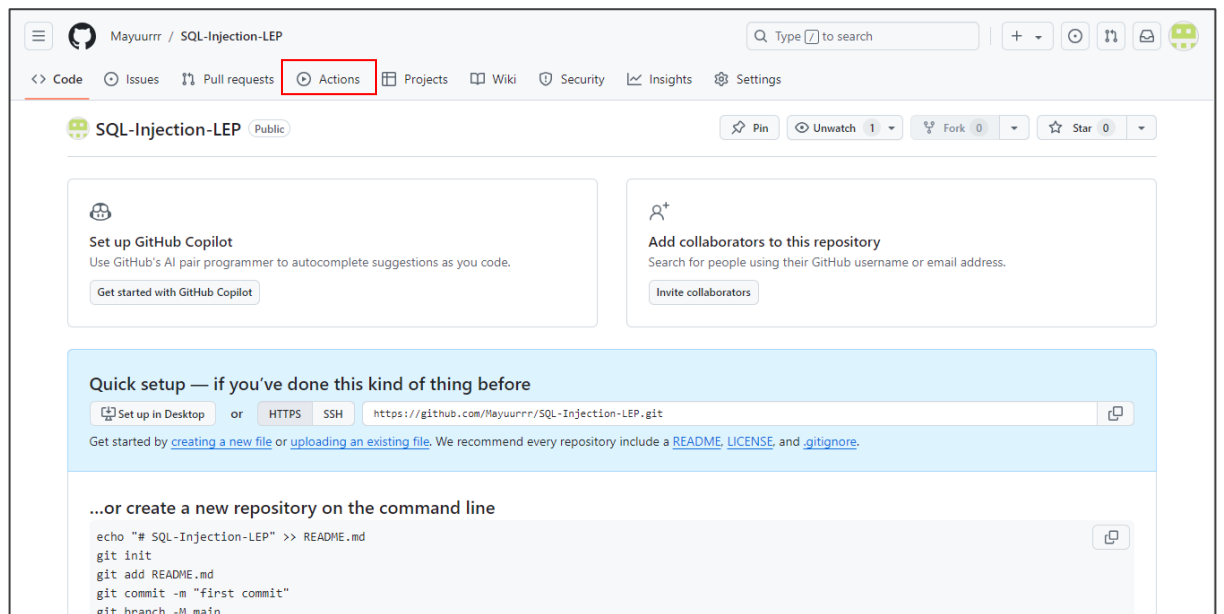
License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

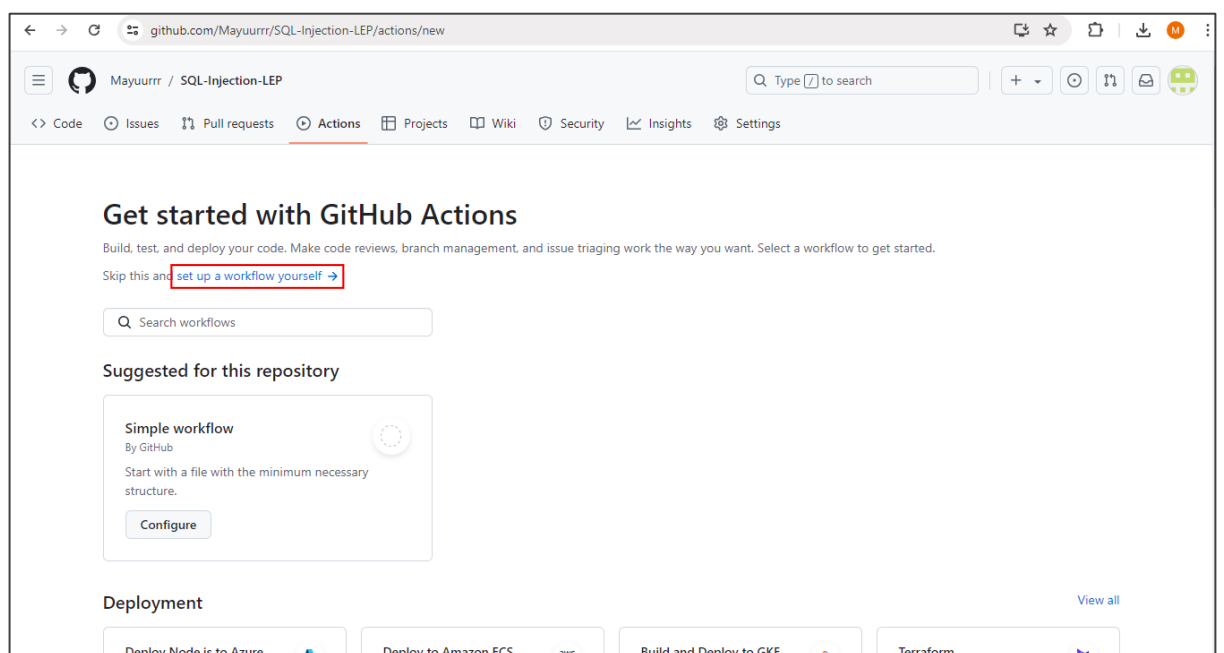
Create repository

© 2024 GitHub, Inc. Terms Privacy Security Status Docs Contact Manage cookies Do not share my personal information

4.5 Navigate to Actions tab

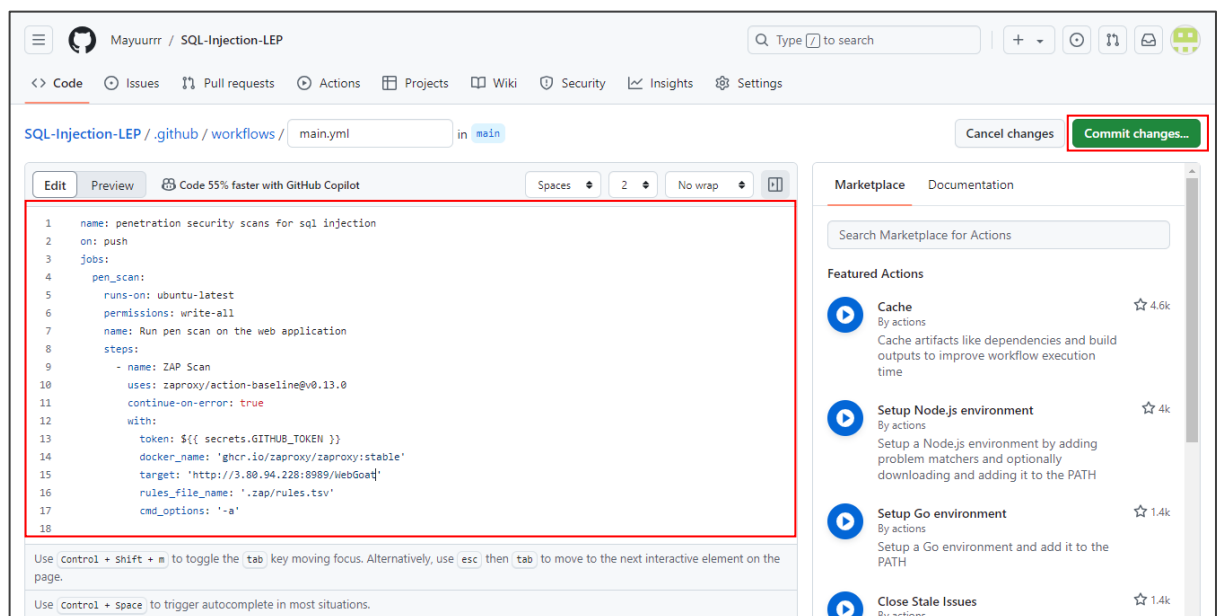


4.6 Click on set up a workflow yourself

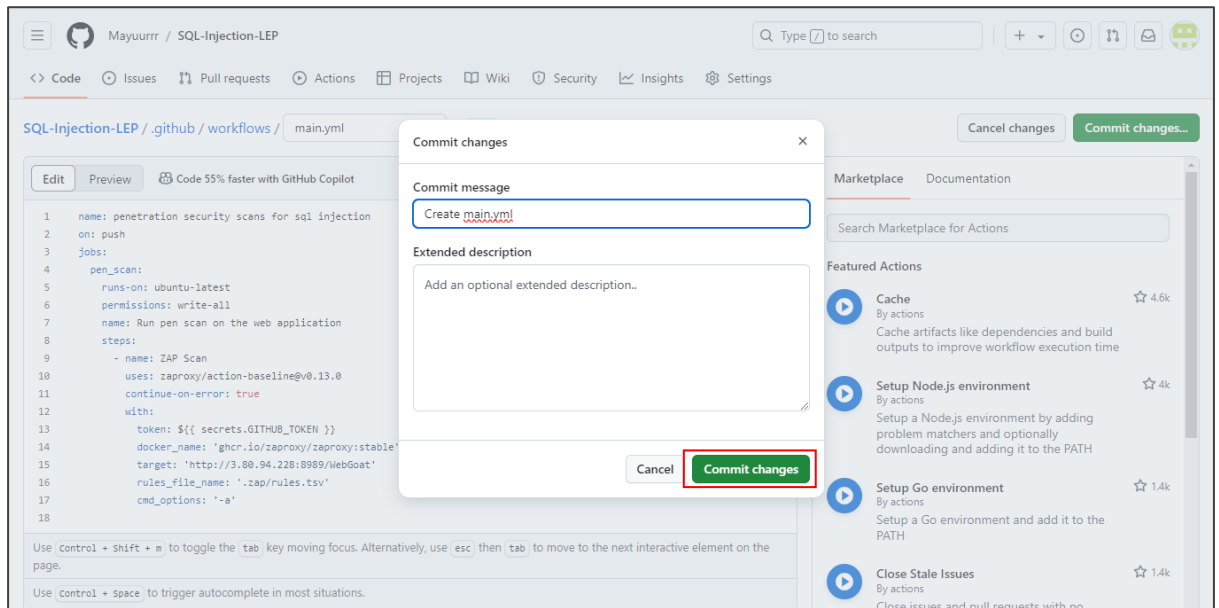


4.7 Paste the code given below in **main.yml** file and click on **Commit changes**

```
name: penetration security scans for sql injection
on: push
jobs:
  pen_scan:
    runs-on: ubuntu-latest
    permissions: write-all
    name: Run pen scan on the web application
    steps:
      - name: ZAP Scan
        uses: zaproxy/action-baseline@v0.13.0
        continue-on-error: true
        with:
          token: ${{ secrets.GITHUB_TOKEN }}
          docker_name: 'ghcr.io/zaproxy/zaproxy:stable'
          target: 'http:// 34.207.181.182/:8989/WebGoat'
          rules_file_name: '.zap/rules.tsv'
          cmd_options: '-a'
```

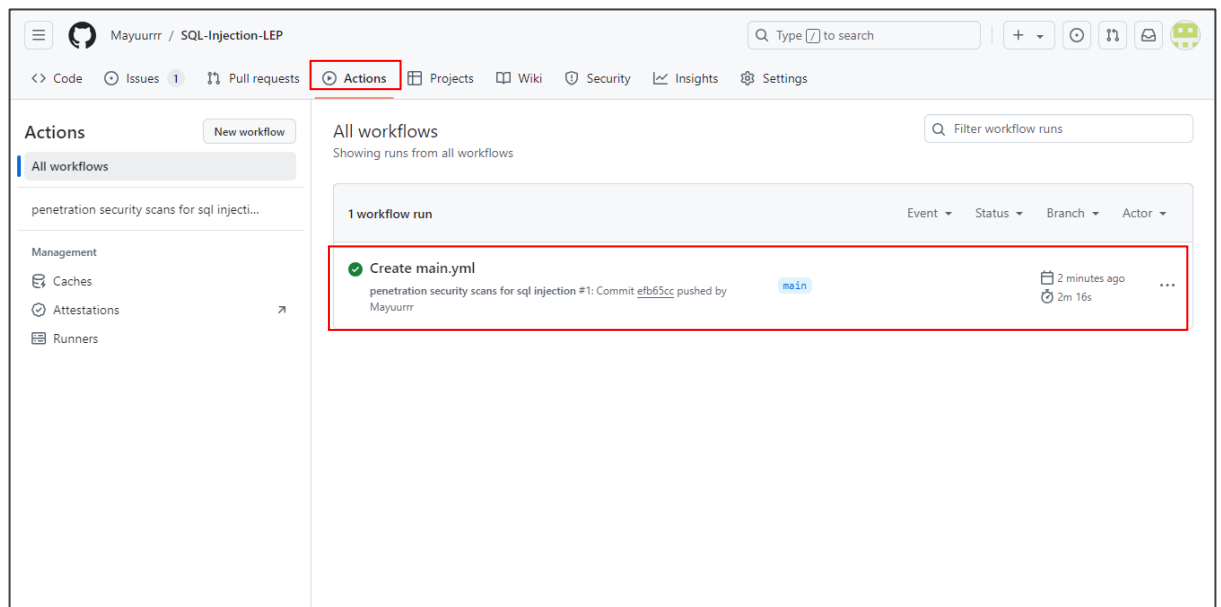


4.8 Click on **Commit changes** to save the file

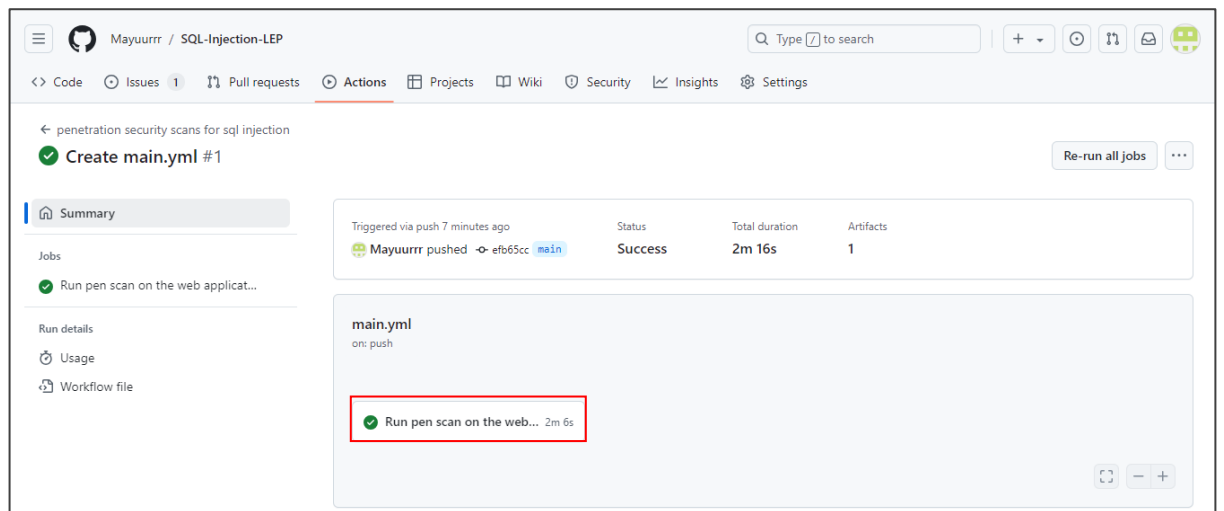


Step 5: Run Workflow and review results

5.1 Navigate to the **Actions** tab and click on the **workflow run** created



5.2 Click on **Run pen scan on the webgoat** to view the details of the scan



The screenshot shows the GitHub Actions interface for a workflow named 'penetration security scans for sql injection'. The workflow has a job named 'Create main.yml #1' which is in a 'Success' state. The job was triggered via a push 7 minutes ago and has a total duration of 2m 16s. The job summary shows a single step: 'Run pen scan on the web applicat...' which is also in a 'Success' state and took 2m 6s to complete. The job is highlighted with a red box.

penetration security scans for sql injection

✓ Create main.yml #1

Re-run all jobs

Summary

Jobs

✓ Run pen scan on the web applicat...

Run details

Usage

Workflow file

Triggered via push 7 minutes ago

Status: Success

Total duration: 2m 16s

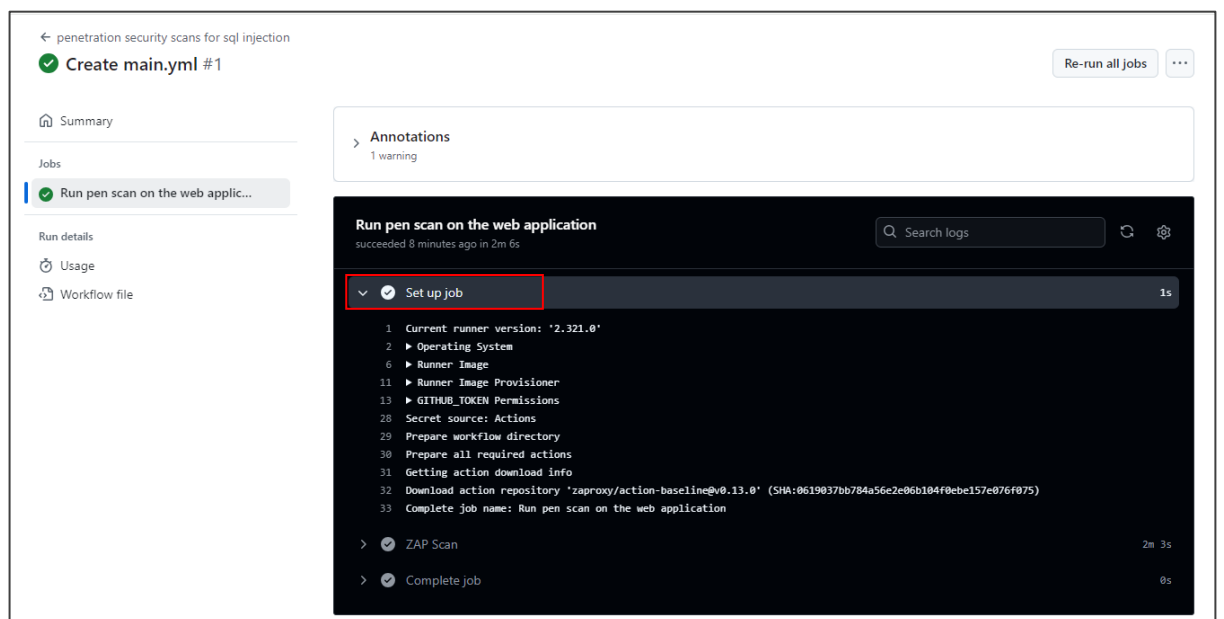
Artifacts: 1

main.yml

on: push

✓ Run pen scan on the web... 2m 6s

5.3 Click on **Set up job** to view the run details



The screenshot shows the GitHub Actions interface for a workflow named 'penetration security scans for sql injection'. The workflow has a job named 'Create main.yml #1' which is in a 'Success' state. The job summary shows a single step: 'Run pen scan on the web applicat...' which is also in a 'Success' state and took 2m 6s to complete. The job is highlighted with a red box. The 'Set up job' step is expanded, showing the following logs:

```
1 Current runner version: '2.321.0'
2 Operating System
6 Runner Image
11 Runner Image Provisioner
13 GITHUB_TOKEN Permissions
28 Secret source: Actions
29 Prepare workflow directory
30 Prepare all required actions
31 Getting action download info
32 Download action repository 'zapproxy/action-baseline@v0.13.0' (SHA:0619037bb784a56e2e06b104f0ebe157e076f075)
33 Complete job name: Run pen scan on the web application
```

The 'Set up job' step is highlighted with a red box.

penetration security scans for sql injection

✓ Create main.yml #1

Re-run all jobs

Summary

Jobs

✓ Run pen scan on the web applicat...

Run details

Usage

Workflow file

Annotations

1 warning

Run pen scan on the web application

succeeded 8 minutes ago in 2m 6s

Search logs

Set up job 1s

1 Current runner version: '2.321.0'

2 Operating System

6 Runner Image

11 Runner Image Provisioner

13 GITHUB_TOKEN Permissions

28 Secret source: Actions

29 Prepare workflow directory

30 Prepare all required actions

31 Getting action download info

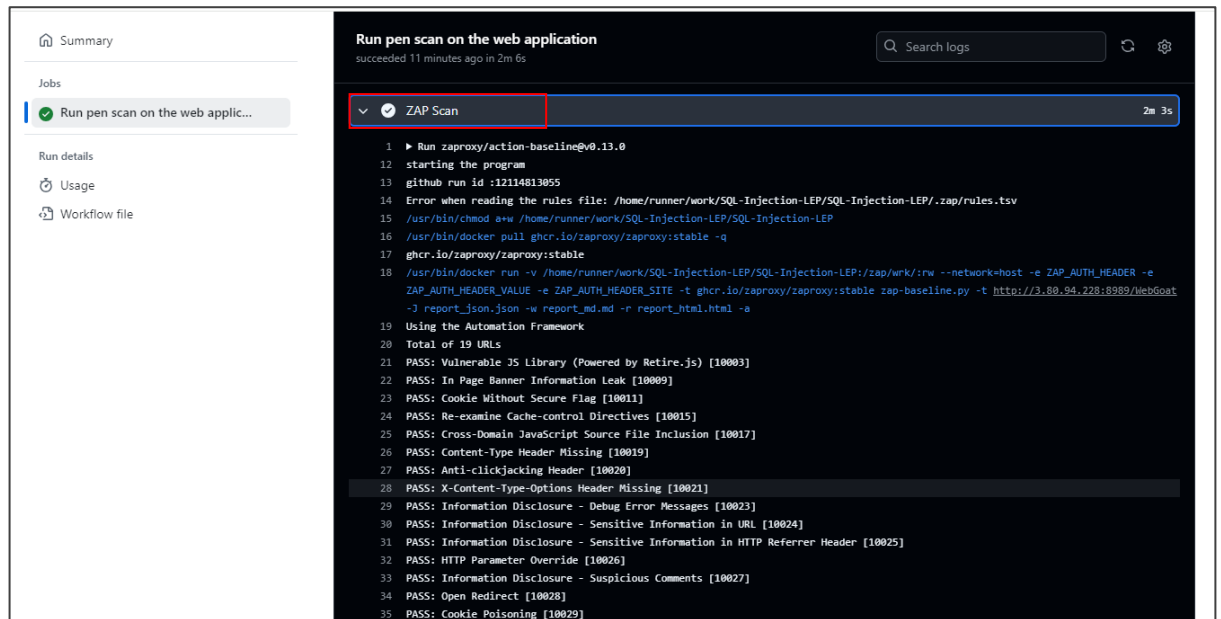
32 Download action repository 'zapproxy/action-baseline@v0.13.0' (SHA:0619037bb784a56e2e06b104f0ebe157e076f075)

33 Complete job name: Run pen scan on the web application

> ZAP Scan 2m 3s

> Complete job 0s

5.4 Click on **ZAP Scan** to view its complete report



The screenshot displays the GitHub Actions interface for a workflow named 'Run pen scan on the web application'. The left sidebar shows the 'Jobs' section with a green checkmark and the job name 'Run pen scan on the web applic...'. Below this, 'Run details' are visible, including 'Usage' and 'Workflow file'. The main panel shows the 'ZAP Scan' job, which succeeded 11 minutes ago. The job log is displayed in a dark theme, showing the execution of ZAP (Zed Attack Proxy) to perform a security scan on a web application. The log includes commands for setting up the environment, pulling the ZAP Docker image, and running the scan. The scan results are listed as a series of 'PASS' messages, indicating that no vulnerabilities were found. The results are as follows:

Line	Message	Severity
20	Total of 19 URLs	
21	PASS: Vulnerable JS Library (Powered by Retire.js) [10003]	
22	PASS: In Page Banner Information Leak [10009]	
23	PASS: Cookie Without Secure Flag [10011]	
24	PASS: Re-examine Cache-control Directives [10015]	
25	PASS: Cross-Domain JavaScript Source File Inclusion [10017]	
26	PASS: Content-Type Header Missing [10019]	
27	PASS: Anti-clickjacking Header [10020]	
28	PASS: X-Content-Type-Options Header Missing [10021]	
29	PASS: Information Disclosure - Debug Error Messages [10023]	
30	PASS: Information Disclosure - Sensitive Information in URL [10024]	
31	PASS: Information Disclosure - Sensitive Information in HTTP Referrer Header [10025]	
32	PASS: HTTP Parameter Override [10026]	
33	PASS: Information Disclosure - Suspicious Comments [10027]	
34	PASS: Open Redirect [10028]	
35	PASS: Cookie Poisoning [10029]	

By following these steps, you have successfully set up an EC2 instance, deployed a vulnerable application, tested SQL Injection vulnerabilities, and integrated automated penetration testing using GitHub Actions. This process demonstrates how to identify and address security risks in a DevSecOps pipeline, enhancing application security through continuous monitoring and automation.